

Headless WordPress Onboarding Tool — Team Handoff

Status: Production-ready (v0.1.0) **Repo:** github.com/Meegrowlabs/headless-wp-onboard **Proof of concept:** ecolive.in **Date:** 15 May 2026

1. What problem this solves

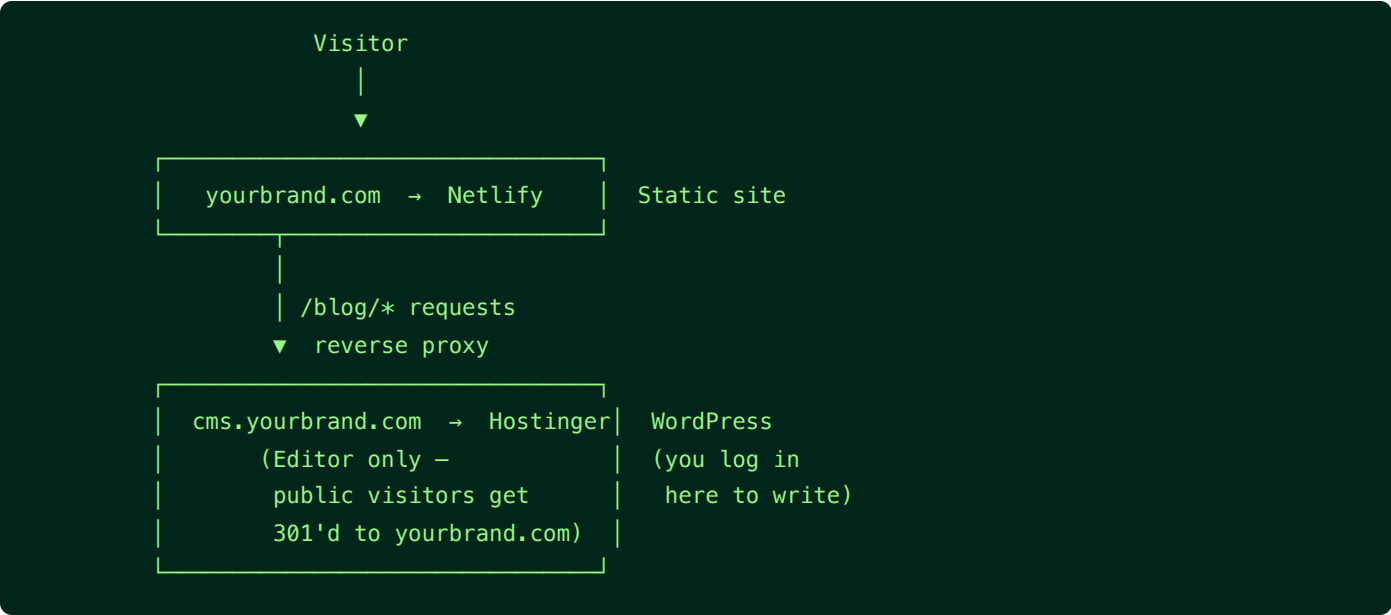
When a new client wants a website with a blog, we used to:

1. Build a static site (HTML/CSS)
2. Set up WordPress somewhere
3. Manually configure DNS
4. Manually edit a dozen WordPress settings
5. Manually install plugins to fix SEO
6. Manually copy blog content over
7. Manually generate Netlify config

That took **3–6 hours** per client and was error-prone — every step has a quirk that the next person doesn't remember.

This tool does steps 3–7 automatically in **about 30 minutes**, and bakes in every weird bug we hit on the ecolive.in build so the team never re-discovers them.

2. Architecture installed



- Visitors only see `yourbrand.com/blog/...` URLs — clean and SEO-friendly
- The team writes in WordPress at `cms.yourbrand.com/wp-admin`
- New posts appear live within seconds (no rebuild needed — proxy fetches latest)
- Canonical URLs, sitemap, robots.txt all served from the primary domain

3. What you can do with the tool

Three ways to drive it

Method	Command	Best for
A. CLI (recommended)	<code>npx headless-wp-onboard init</code>	You/teammate setting up directly
B. AI agent playbook	Open <code>PLAYBOOK.md</code> , tell Claude/Codex/Cursor "follow this"	Working through an AI assistant
C. Template repo	Copy <code>starter-template/</code> for new client's static site	Starting a brand new site from scratch

Commands the CLI supports

```
# Full guided setup – runs everything (10-12 questions)
headless-wp-onboard init

# Or run individual pieces:
headless-wp-onboard configure-wp      # Set WP options (title, URLs, permalinks)
headless-wp-onboard install-plugin    # Build & upload SEO redirect plugin
headless-wp-onboard migrate --posts posts.json
headless-wp-onboard dns                # Provision NS records for cms subdomain
headless-wp-onboard generate-files     # Write netlify.toml + robots.txt
headless-wp-onboard verify             # Run 7 smoke tests
```

4. What's automated vs manual

Step	Status	Where
Buy domain	✗ Manual	Any registrar
Hostinger account + WP install	✗ Manual	hPanel UI
Copy Hostinger NS pair	✗ Manual	Hostinger UI
Configure WP URLs and permalinks	✓ Auto	XML-RPC
Set site title / tagline	✓ Auto	XML-RPC
Build & upload SEO redirect plugin	✓ Auto	REST API
Activate plugin	✓ Auto	REST API
Purge LiteSpeed cache	✓ Auto	admin-ajax
DNS — Cloudflare	✓ Auto	Cloudflare API
DNS — Netlify	✓ Auto	Netlify API
DNS — GoDaddy / Hostinger	✗ Manual	Prints exact records
Bulk migrate posts	✓ Auto	XML-RPC
Generate netlify.toml	✓ Auto	Template render
Generate robots.txt	✓ Auto	Template render
Smoke tests	✓ Auto	fetch()

5. Prerequisites

What the client (or you) must do BEFORE running the tool

1. **Buy a domain** at any registrar.
2. **Create a Hostinger account** and install WordPress at `cms.<domain>` (e.g. `cms.yourbrand.com`).

3. From Hostinger hPanel → your WP site → Domain → Connect, **copy the nameserver pair** shown (e.g. `aurora.dns-parking.com` / `nebula.dns-parking.com`).

4. **Decide where DNS is managed** for the primary domain:

- Cloudflare → get an API token (My Profile → API Tokens → Create → "Edit zone DNS" template, scoped to the domain's zone) - Netlify DNS → get a personal access token (User settings → Applications → Personal access tokens → New) - GoDaddy / Hostinger / other → manual — the tool prints exact records to paste

1. The **static site repo** must exist (a basic HTML site). If not, copy `starter-template/` from this tool.

What you need locally

- **Node.js 18+** (check with `node -v`)
- **zip command** (preinstalled on macOS and most Linux)
- **git and gh** (GitHub CLI) — optional but useful
- A clone of the static-site repo for the client

6. How to test it — full walkthrough

This is the script for a teammate to follow on a fresh client. Pick any test domain you control. If you don't want to use a real domain yet, you can do a "dry run" by pointing only the cms subdomain to a test Hostinger site.

Step 0 — Install the tool

```
git clone git@github.com:Meegrowlabs/headless-wp-onboard.git
cd headless-wp-onboard
npm install
```

(Or, once published to npm: `npx headless-wp-onboard init`)

Step 1 — Prepare the WordPress side

Manually in Hostinger hPanel:

1. Add new website → choose subdomain (e.g. `cms.testbrand.com`)

2. Install WordPress
3. Set admin email/password (you'll need these soon)
4. Go to Domain → Connect, copy the **nameserver pair** (looks like `aurora.dns-parking.com` and `nebula.dns-parking.com`)

Step 2 — Run the CLI

```
node bin/cli.js init
```

You'll be asked:

Prompt	Sample answer
Primary domain	<code>testbrand.com</code>
CMS subdomain	<code>cms.testbrand.com</code> (default)
Blog path	<code>/blog</code> (default)
Site title	<code>Testbrand Insights</code>
Site tagline	<code>Our latest thinking</code>
Brand name	<code>Testbrand</code>
Is WP installed?	Yes
WP admin email	<code>admin@testbrand.com</code>
WP password	(hidden)
WP app password	(leave empty first time — generate later)
DNS provider	Cloudflare / Netlify DNS / Manual
API token (if auto DNS)	(paste it)
Hostinger NS 1	<code>aurora.dns-parking.com</code>
Hostinger NS 2	<code>nebula.dns-parking.com</code>
Posts JSON path	(empty to skip)
Static-site repo path	<code>../testbrand-website</code>

The CLI saves all answers to `./client.json` . **Add this file to .gitignore** — it contains passwords.

Step 3 — What happens during init

```
[1/6] Verifying WordPress credentials      ✓
[2/6] Configuring WordPress                ✓
      title → "Testbrand Insights"
      permalink → /blog/%postname%/
      home → testbrand.com
      admin URL → cms.testbrand.com
[3/6] Installing SEO redirect plugin      (skipped — no app password)
[4/6] Provisioning DNS                    ✓ (Cloudflare NS records added)
[5/6] Migrating posts                     (skipped)
[6/6] Generating Netlify config files      ✓
      ../testbrand-website/netlify.toml
      ../testbrand-website/robots.txt
```

Step 4 — Generate WP application password

In `cms.testbrand.com/wp-admin` → Users → Profile → scroll to Application Passwords → enter name "claude-migration" → Add. Copy the shown password. Paste it into `client.json` as `wp.app_password`.

 **Gotcha:** if you see "Cookie check failed" when clicking Add, temporarily go to Settings → General and change "Site Address" back to `https://cms.testbrand.com`, save, generate the app password, then change Site Address back to `https://testbrand.com`.

Step 5 — Run plugin install separately

```
node bin/cli.js install-plugin
```

This will:

- Build a `.zip` with the SEO redirect plugin (rendered from the template with your domain values)
- Upload + activate via the WP REST API
- Purge LiteSpeed cache automatically

Step 6 — Commit and push the static site

```
cd ../testbrand-website
git add -A
git commit -m "Add headless WordPress + Netlify proxy config"
git push
```

Netlify will deploy in ~30 seconds (no build step needed).

Step 7 — Wait for DNS, then verify

DNS propagation: 5–30 minutes for the cms subdomain. Watch with:

```
dig cms.testbrand.com +short
```

When you see an IP returned, run:

```
cd ../headless-wp-onboard
node bin/cli.js verify
```

You should see:

```
✓ Direct cms.testbrand.com blog URL should 301 – 301 → https://testbrand.com/...
✓ Proxied URL via testbrand.com should 200 – 200 OK
✓ Page title contains site title
✓ Canonical URL is on primary domain
✓ wp-admin still accessible on cms subdomain
✓ Sitemap reachable on primary domain
✓ robots.txt on primary domain

🇺🇸 7 passed, 0 failed
```

If any check fails, see the "Known gotchas" section in [PLAYBOOK.md](#).

Step 8 — Migrate existing posts (optional)

Build a `posts.json` file like this:


```
[
  {
    "slug": "url-friendly-slug",
    "title": "Article Title",
    "date": "2026-05-15",
    "category": "Strategy",
    "excerpt": "Short summary used as meta description.",
    "content": "<p>HTML body of the post...</p><h2>Subheading</h2><p>...</p>"
  }
]
```

Then:

```
node bin/cli.js migrate --posts posts.json
```

Each post posts via XML-RPC with a progress bar.

Step 9 — Hand it over to the client

- Tell them the new login URL: `cms.testbrand.com/wp-admin`
- Have them **change the WordPress password** (the one in `client.json` is in plain text in your repo's gitignored file — treat as compromised)
- Delete `client.json` or move it to a password manager when done
- The blog at `testbrand.com/blog` is now live, dynamic, and SEO-clean

7. Smoke test checklist (the 7 verify checks explained)

Each check is a curl against the live site. The verify command runs them all in ~10 seconds.

#	Check	Expected	Why it matters
1	<code>cms.<domain>/blog/X</code> returns 301	Yes	Direct visitors must bounce to primary domain, not see the cms host
2	<code><domain>/blog/X</code> returns 200	Yes	The proxy must serve content under primary domain
3	<code><title></code> contains site title	"Brand Insights"	Page titles look professional in search results
4	<code><link rel=canonical></code> points to primary domain	<code>https://<domain>/blog/X/</code>	Google indexes only the primary domain
5	<code>cms.<domain>/wp-admin</code> is reachable	302 → login	You can still log in to edit
6	<code><domain>/wp-sitemap.xml</code> returns 200	Yes	Search engines can find all posts
7	<code><domain>/robots.txt</code> references the sitemap	Yes	Search engines find the sitemap via robots.txt

If **all 7 pass**, the setup is fully working. Hand it to the client.

8. Known gotchas (read before debugging)

These are bugs we hit during the *ecolive.in* build. The tool handles them — but if something weird happens, these are your usual suspects.

"Cookie check failed" when creating WP application passwords

WordPress sets login cookies based on the home URL. If `home = primary domain` but the admin UI is at `cms.subdomain`, the cookie domains mismatch and AJAX form submissions fail silently with this notice.

Fix: Temporarily set Site Address back to `cms.<domain>` , generate the app password, then restore. The tool's `install-plugin` command uses REST API upload (not the AJAX UI) to sidestep this entirely.

LiteSpeed Cache serves stale HTML

Hostinger ships LiteSpeed Cache enabled by default. Changes to permalinks, site title, or plugin behavior won't appear until cache is purged.

Fix: WP admin → LiteSpeed Cache → Toolbox → Purge All. The CLI does this automatically after plugin install, but if you make manual changes in WP admin, purge manually.

WordPress canonical redirect breaks proxied `/wp-sitemap.xml`

WordPress's `redirect_canonical` core function redirects any URL accessed via a non-canonical host. When Netlify proxies `cms.<domain>/wp-sitemap.xml` , WP returns 301 to `<domain>/wp-sitemap.xml` , which proxies back to cms → infinite loop.

Fix: The SEO redirect plugin disables `redirect_canonical` for `/wp-sitemap` and `/feed` paths. Baked into the template.

Netlify edge cache holds stale 301s

After fixing a redirect, Netlify edge may still serve the old response for a few minutes.

Fix: Verify uses cachebust query params (`?_=<timestamp>`) on cached paths. For visitors it self-resolves within minutes.

Hostinger plugin search returns empty

Hostinger's WP plugin search sometimes fails to return results for common plugins.

Fix: Upload plugins via the .zip uploader instead. The CLI uses REST API upload, which works around this.

Cloudflare "no zone found"

The domain must be on Cloudflare AND the API token must have permission for that zone.

Check: `curl https://api.cloudflare.com/client/v4/zones?name=<domain> -H "Authorization: Bearer <token>"` — should return the zone in `result` .

9. What to do if init fails midway

The tool is **idempotent** — you can re-run any subcommand without breaking anything.

- WP misconfigured → `node bin/cli.js configure-wp`
- Plugin missing → `node bin/cli.js install-plugin`
- DNS not done → `node bin/cli.js dns`
- Posts didn't migrate → `node bin/cli.js migrate --posts posts.json`
- Files not generated → `node bin/cli.js generate-files`
- Want to check everything → `node bin/cli.js verify`

Each command reads `client.json` for config. If something needs to be redone, delete the stale state in WP (or undo the change manually) and re-run.

10. Roadmap (future improvements)

Suggested next features when we have time:

Priority	Feature	Effort
High	Auto-create Netlify site via Netlify API	2 hrs
High	<code>migrate-from-html</code> to scrape existing static HTML into <code>posts.json</code>	3 hrs
Medium	Build hook setup (WP publish → Netlify rebuild) — optional but helpful for SSG fallback	2 hrs
Medium	Pre-flight check: validate Hostinger reachability before asking for password	30 min
Low	TUI dashboard showing all steps in real time	4 hrs
Low	Hostinger SSH/cPanel scrape to auto-install WP (currently manual)	8 hrs

11. Team contacts

- **Repo:** <https://github.com/Meegrowlabs/headless-wp-onboard>
- **Reference site:** <https://ecolive.in> (built with this exact stack)

- **Documentation:** see [PLAYBOOK.md](#) in the repo for the agent-facing version

Appendix A — Sample `client.json` schema

```
{
  "domain": "yourbrand.com",
  "cms_domain": "cms.yourbrand.com",
  "blog_path": "/blog",
  "site_title": "Yourbrand Insights",
  "site_tagline": "Our latest thinking",
  "brand": "Yourbrand",
  "wp": {
    "user": "admin@yourbrand.com",
    "pass": "(actual password)",
    "app_password": "(application password)"
  },
  "dns": {
    "provider": "cloudflare",
    "token": "(Cloudflare API token)"
  },
  "hostinger_ns": [
    "aurora.dns-parking.com",
    "nebula.dns-parking.com"
  ],
  "posts_file": "./posts.json",
  "site_repo_path": "../yourbrand-website"
}
```

Always `.gitignore` this file. It contains passwords.

Appendix B — Architecture cheatsheet for the team

When a teammate asks "wait, how does the blog work?" — this is the one-paragraph answer:

Visitors hit `yourbrand.com/blog/post-name` . Netlify sees a `/blog/*` request and reverse-proxies it to `cms.yourbrand.com/blog/post-name` — WordPress generates the HTML there. The HTML's canonical URL says `yourbrand.com` , so Google indexes only the primary domain. If anyone visits `cms.yourbrand.com/blog/post-name` directly, our SEO redirect plugin sends them to the primary domain with a 301. The team writes posts in `cms.yourbrand.com/wp-admin` . There is no build step — new posts go live instantly via the proxy.